

Python Programming Fundamentals

Classes and Objects

Programming Fundamentals Team

SETU

2026-11-11



Outline

1. What is OOP?
2. Classes vs Objects
3. Class Definition Syntax
4. The `__init__` Method and `self`
5. Instance Methods
6. The `__str__` Method
7. A Complete Example — Student Class



Outline

1. What is OOP?
2. Classes vs Objects
3. Class Definition Syntax
4. The __init__ Method and self
5. Instance Methods
6. The __str__ Method
7. A Complete Example — Student Class



1. What is OOP?

Object-Oriented Programming (OOP) organises code around **objects** — things that have both **state** (data) and **behaviour** (functions).



1. What is OOP?

Object-Oriented Programming (OOP) organises code around **objects** — things that have both **state** (data) and **behaviour** (functions).

Procedural programming (what you've done so far):

- Data stored in variables
- Functions operate on data passed as arguments



1. What is OOP?

Object-Oriented Programming (OOP) organises code around **objects** — things that have both **state** (data) and **behaviour** (functions).

Procedural programming (what you've done so far):

- Data stored in variables
- Functions operate on data passed as arguments

Object-Oriented programming:

- Data and the functions that work on it are **bundled together** in objects
- Objects are created from **classes** (blueprints)
- Easier to model real-world entities



1. What is OOP?

Object-Oriented Programming (OOP) organises code around **objects** — things that have both **state** (data) and **behaviour** (functions).

Procedural programming (what you've done so far):

- Data stored in variables
- Functions operate on data passed as arguments

Object-Oriented programming:

- Data and the functions that work on it are **bundled together** in objects
- Objects are created from **classes** (blueprints)
- Easier to model real-world entities

Real-world analogy: a `BankAccount` has state (balance, account number) and behaviour (deposit, withdraw, check balance).



Outline

1. What is OOP?

2. Classes vs Objects

3. Class Definition Syntax

4. The __init__ Method and self

5. Instance Methods

6. The __str__ Method

7. A Complete Example — Student Class



2. Classes vs Objects

Class — the blueprint or template. Defined once.

Object (instance) — a specific example created from the blueprint. You can create many.

```
# 'Dog' is the class (blueprint)
class Dog:
    pass

# Each of these is a separate object (instance)
rex   = Dog()
buddy = Dog()
fido  = Dog()

print(type(rex))          # <class '__main__.Dog'>
print(rex is buddy)       # False — different objects
```



2. Classes vs Objects

Analogy: the class `Dog` is like the concept of "dog" — the objects `rex`, `buddy`, `fido` are specific individual dogs.



Outline

1. What is OOP?

2. Classes vs Objects

3. Class Definition Syntax

4. The __init__ Method and self

5. Instance Methods

6. The __str__ Method

7. A Complete Example — Student Class



3. Class Definition Syntax

```
class ClassName:
    """Docstring describing the class."""

    def __init__(self, param1, param2):
        """Initialiser — runs when object is created."""
        self.attribute1 = param1
        self.attribute2 = param2

    def method_name(self):
        """A method — a function defined in the class."""
        # can access self.attribute1, self.attribute2, etc.
```



3. Class Definition Syntax

Key rules:

- Class names use `PascalCase` (each word capitalised)
- `__init__` is the **constructor** — called automatically when you create an object
- Every method's first parameter is `self` — a reference to the object itself
- `self.something` creates an **instance variable** — stored on the object



Outline

1. What is OOP?

2. Classes vs Objects

3. Class Definition Syntax

4. The __init__ Method and self

5. Instance Methods

6. The __str__ Method

7. A Complete Example — Student Class



4. The __init__ Method and self

```python
class Person:
    def __init__(self, name, age):
        self.name = name    # instance variable
        self.age  = age     # instance variable

# Creating objects — __init__ is called automatically
alice = Person("Alice", 25)
bob   = Person("Bob", 30)
```



4. The __init__ Method and self

```
# Accessing instance variables
print(alice.name)   # Alice
print(bob.age)      # 30

# Each object has its OWN copy of the variables
alice.age = 26      # only changes alice's age
print(alice.age)    # 26
print(bob.age)      # 30  (unchanged)
```



Outline

1. What is OOP?

2. Classes vs Objects

3. Class Definition Syntax

4. The __init__ Method and self

5. Instance Methods

6. The __str__ Method

7. A Complete Example — Student Class



5. Instance Methods

Instance methods are functions defined inside a class. They always receive `self` as the first argument.

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        """Return the area of the circle."""
        return 3.14159 * self.radius ** 2

    def circumference(self):
        """Return the circumference."""
        return 2 * 3.14159 * self.radius
```



5. Instance Methods

```
    def scale(self, factor):
        """Scale the circle by a factor."""
        self.radius *= factor

c = Circle(5)
print(f"Area: {c.area():.2f}")           # 78.54
print(f"Circumference: {c.circumference():.2f}")  # 31.42

c.scale(2)
print(f"After scaling, radius = {c.radius}")  # 10
```



Outline

1. What is OOP?

2. Classes vs Objects

3. Class Definition Syntax

4. The __init__ Method and self

5. Instance Methods

6. The __str__ Method

7. A Complete Example — Student Class



6. The __str__ Method

__str__ defines what print() and str() display for your object.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age  = age

    def __str__(self):
        """Return a human-readable string representation."""
        return f"Person(name='{self.name}', age={self.age})"
```



6. The __str__ Method

```
# contd...

alice = Person("Alice", 25)
print(alice)          # Person(name='Alice', age=25)
print(str(alice))     # Person(name='Alice', age=25)
                      # You don't need to call str() explicitly;
                      # print() does it for you.

# With __str__, you get something like:
# Person(name='Alice', age=25)

# Without __str__, you'd get something like:
# <__main__.Person object at 0x7f...>
```



6. The __str__ Method

```
# contd...

alice = Person("Alice", 25)
print(alice)          # Person(name='Alice', age=25)
print(str(alice))     # Person(name='Alice', age=25)
                      # You don't need to call str() explicitly;
                      # print() does it for you.

# With __str__, you get something like:
# Person(name='Alice', age=25)

# Without __str__, you'd get something like:
# <__main__.Person object at 0x7f...>
```

Best practice: always define `__str__` on your classes. It makes debugging much easier.



Outline

1. What is OOP?

2. Classes vs Objects

3. Class Definition Syntax

4. The __init__ Method and self

5. Instance Methods

6. The __str__ Method

7. A Complete Example — Student Class



7. A Complete Example — Student Class

```
class Student:
    """Represents a student with a name, ID, and list of scores."""

    def __init__(self, name, student_id):
        self.name       = name
        self.student_id = student_id
        self.scores     = []

    def add_score(self, score):
        """Add a score to this student's record."""
        self.scores.append(score)
```



7. A Complete Example — Student Class

```
    def average(self):
        """Return average score, or 0 if no scores."""
        if not self.scores:
            return 0
        return sum(self.scores) / len(self.scores)

    def is_passing(self, threshold=40):
        """Return True if the student's average meets the threshold."""
        return self.average() >= threshold

    def __str__(self):
        avg = self.average()
        status = "Pass" if self.is_passing() else "Fail"
        return f"[{self.student_id}] {self.name}: avg={avg:.1f}
({status})"
```



7. A Complete Example — Student Class

```
s = Student("Alice", "S001")

s.add_score(85)
s.add_score(92)
s.add_score(78)

print(s)    # [S001] Alice: avg=85.0 (Pass)
```

Thanks for Watching - Any questions?

